Verified compiler for a language with control effects

(Weryfikacja kompilatora dla języka programowania z efektami sterowania)

Paweł Dybiec

Praca magisterska

Promotor: Piotr Polesiuk

Uniwersytet Wrocławski
Wydział Matematyki i Informatyki
Instytut Informatyki

9 marca 2026

Abstract

We present a type system for a language with labeled delimited control operators. It utilizes polymorphic types and effects to allow expressions parametrized by effects. Our goal is to formally prove correctness of such language and present translation of it to simple abstract machine.

Tematem tej pracy jest stworzenie i zweryfikowanie języka z operatorami kontroli dla kontynuacji ograniczonych. Używa on polimorficznych typów aby pozwolić na wyrażenie obliczeń generycznych wobec pewnych efektów. Celem pracy jest zweryfikowanie poprawnosci definicji takiego języka oraz jego translacji do maszyny abstrakcyjnej.

Contents

Chapter 1

Introduction

Control operators have been present in languages for a long time, one notable example is SCHEME programming language which started in 70s[1]. They enable non-local jumps, that might be familiar to users of imperative languages with goto operator. But those operators unlinke goto, are handled by capturing continuation, and storing it as first-class value which can be passed, wrapped or dropped.

One issue one might have with call/cc operator is that when called, it catches whole rest of program in the continuation.Operators for delimited control introduced by Danvy, Filinski[2] and Felleisen[3] allow to control continuations them more precisely. Operator reset delimits continuations caught by shift. This allows for localized use of control effects, scope of captured continuation is much more visible. While they still allow for non-local execution, which enables intresting computational effects such as emulating non-determinism, exceptions, failure and others.

In this thesis we try to build typed version of such calculus with polymorphic types (similiar to Piróg, Polesiuk and Sieczkowski[4]), but for multipromt delimited continuations. Usually such calculi jump from shift to closest enclosing reset. Here we tried labeling them to allow stacking of different effects.

Formalisation work was done in Agda 2.8. Code archive has also nix derivation describing exact environment to build both package and this paper to ensure reproducibility.

Chapter 2

Surface language

2.1 Kinds, Types, Effects and Expressions

Variable and type variables are de Bruijn indices.

```
data Kind : Set where
  T : Kind
  E : Kind
```

Types and Effects are defined mutally recursive. Type is either variable, arrow, forall or label. Effect is list of types, it's used in arrow and forall constructors of type to keep effects of computation underneath and it will be used in typing judgement. Label contructor just stores type variable bound to effect represented by the label and then Type and Effect of delimited computation.

```
module Types where
  Id : Set
  Id = ℕ
  data Type : Set
  Effects : Set
  data Type where
    ttv : Id → Type
    _-_>_ : Type → Effects → Type → Type
    forallt : Kind → Type → Effects → Type
    L_at_/_ : Type → Type → Effects → Type
  Effects = List Type
open Types
```

Constructors of expressions such as `var`, `lam`, `app`, `tlam`, `tapp` behave as usual. `var` is de Bruijn indexed variable, `lam` is lambda abstraction, `app` is function application, `tlam` is type abstraction, but it stores Kind of abstracted type and `tapp` is type application. The rest of constructors are responsible for continuations and labels. Constructor `new` bind label that is used by shift and reset constructors to pair up. $\texttt{shift}_0$ stores label and expression (parametrized by continuation) that replaces whole delimited computation. Last constructor of expressions is $\texttt{reset}_0$ which has 3 fields, last one is label that is used to match up reset with shifts. First argument is delimited computation where $\texttt{shift}_0$ can be used. So when $\texttt{shift}_0$ `k.e` is being evaluated it aborts enclosing corresponding reset, instead of it now

`e` is being evaluated with `k` bound to continuation representing evaluation context between shift and it's reset. Second field `x. en` is used when during evaluation of first expression no corresponding shift aborts, and just yields value `v` - then whole reset evaluates to `en` where `x` is bound to `v`.

```
module Expr where
  data Expr : Set where
    var : ℕ → Expr
    lam : Expr → Expr
    app : Expr → Expr → Expr
    tlam : Kind → Expr → Expr
    tapp : Expr → Type → Expr
    new : Expr → Expr
    shift₀ : Expr → Expr → Expr
    reset₀ : Expr → Expr → Expr → Expr
open Expr
```

2.2 Contexts

Contexts are represented as list of types, and type contexts are represented as lists of kinds. Judgements for membership of types have Peano numbers structure.

```
module Typing where
  infixl 5 _,_
  data Context : Set where
    ∅ : Context
    _,_ : Context → Type → Context

  data TContext : Set where
    ∅ : TContext
    _,_ : TContext → Kind → TContext
```

2.3 Typing judgements

Expressions are extrinsically typed, thus typing judgements are represented separately. Typing rules for `var`, `lam`, `app`, `tlam`, `tapp` are defined mostly as usual. The noticable difference is that expressions for values are generic over effect they execute.

```
data _,_⊢_∶_/_ : TContext → Context → Expr → Type → Effects → Set where
  ⊢var : ∀ {Γ Δ x A E}
    → Γ ∋ x ∶ A
    ----------------------
    → Δ , Γ ⊢ var x ∶ A / E

  ⊢lam : ∀ {Γ Δ e A B E F}
    → Δ , (Γ , A) ⊢ e ∶ B / E
    ---------------------------------
    → Δ , Γ ⊢ lam e ∶ A - E > B / F

  ⊢weak : ∀ {Γ Δ e A A' E E'}
    → Δ ⊢ A <t∶ A'
    → Δ ⊢ E <∶ E'
    → Δ , Γ ⊢ e ∶ A / E
    --------------------
    → Δ , Γ ⊢ e ∶ A' / E'

  ⊢app : ∀ {Γ Δ e1 e2 A B E}
    → Δ , Γ ⊢ e1 ∶ A - E > B / E
    → Δ , Γ ⊢ e2 ∶ A / E
    ----------------------------
```

```
    → Δ , Γ ⊢ app e1 e2 ∶ B / E

  ⊢forall : ∀ {Γ Δ e k A E F}
    → (Δ , k) , Γ ⊢ e ∶ TypeSubst.bump A / TypeSubst.bump' E
    ------------------------------------
    → Δ , Γ ⊢ tlam k e ∶ forallt k A E / F

  ⊢tapp : ∀ {Γ Δ e k A T E}
    → Δ ⊢ T ∶t
    → Δ , Γ ⊢ e ∶ forallt k A E / E
    ------------------------------------------------
    → Δ , Γ ⊢ tapp e T ∶ A TypeSubst.[ T ] / (E TypeSubst.effs[t T ])
```

`new` introduces new type variable, and variable that represent respectively effect and label. Type of label stores effect bound by label (here `ttv zero`). `A1 / A2` represent type and effect of delimited computation.

```
  ⊢new : ∀ {Γ Δ e A A1 E E1}
    → (Δ , Kind.E) , (Γ , (L ttv zero at A1 / E1)) ⊢ e ∶ TypeSubst.bump A / TypeSubst.bump' E
    ----------------------
    → Δ , Γ ⊢ new e ∶ A / E
```

$\texttt{shift}_0$ uses only one effect `ttv n` that's represented by label. For it to be properly typed expression inside shift bind extra variable - where continuation will be plugged into. So type of this expression should take continuation and returns same type as reset, with effects visible in reset. And continuation itself should be an arrow from type of shift to type of whole delimited computation with effects of same computation. Since continuation passed there will have $\texttt{reset}_0$, that $\texttt{reset}_0$ will introduce effect `ttv n`, so it shouldn't be represented in type of argument.

```
  ⊢shift₀ : ∀ {Γ Δ e e' A A' n E'}
    → Δ ⊢ ttv n ∶e
    → Δ , Γ ⊢ e' ∶ (L ttv n at A' / E') / nil
    → Δ , (Γ , A - E' > A' ) ⊢ e ∶ A' / E'
    ---------------------------------------
    → Δ , Γ ⊢ shift₀ e' e ∶ A / (ttv n :: nil)
```

$\texttt{reset}_0$ has three parameters, first is expression that will have access to effect, so its list of effects is expanded. Second is continuation that will handle value returned from first argument. And third is label, which type stores type and effects of whole expression.

```
  ⊢reset₀ : ∀ {Γ Δ e e' en A A' n E'}
    → Δ ⊢ ttv n ∶e
    → Δ , Γ ⊢ e ∶ A / (ttv n :: E')
    → Δ , (Γ , A) ⊢ en ∶ A' / E'
    → Δ , Γ ⊢ e' ∶ (L ttv n at A' / E') / nil
    ----------------------------------
    → Δ , Γ ⊢ reset₀ e en e' ∶ A' / E'
```

Chapter 3

Runtime language

3.1 Runtime language

Since grammar of terms doesn't have any expression that would have type of label, and shift and reset require same labels, that would mean that reduction relation would need to go under `new` binders. Instead we will define another expression language expanded by notion of label values.

When `new` expression is evaluated then all occurences of variables bound by it would have it replaced with newly allocated label value. That means evaluation would need to keep a state for allocator.

To ensure type safety we will expand syntax of types with allocated effects, and add new effect context to typing judgmenents, that maps allocations to type and effect of delimiter.

```
module Runtime where


module Types_ where
  Id : Set
  Id = ℕ
  Label = ℕ
  data Type : Set
  Effects : Set
  data Type where
    ttv : Id → Type
    _-_>_ : Type → Effects → Type → Type
    forallt : Kind → Type → Effects → Type
    L_at_/_ : Type → Type → Effects → Type
    Effect : Label → Type -- allocated effect
  Effects = List Type
open Types_
```

Most of constructors in `RExpr` are the same as in `Expr`. Labels runtime values are represented by Natural numbers.

```
module RuntimeExpr where
  data RExpr : Set where --runtime version
    var : ℕ → RExpr
    lam : RExpr → RExpr
    app : RExpr → RExpr → RExpr
    tlam : Kind → RExpr → RExpr
```

```
    tapp : RExpr → Type → RExpr
    new : RExpr → RExpr
    shift₀ : RExpr → RExpr → RExpr
    reset₀ : RExpr → RExpr → RExpr → RExpr
```

And here we have separate term for labels, it just stores label identifier.

```
label : Label → RExpr
```

Since grammar of types has changed, we need to redefine contexts. Unfortunately almost all of typing judgements need to be redefined. Here we also introduce new context for effects, it's just finite vector of types and effects. Since they are only used during evaluation, the types and effects have no free variables.

```
module Typing where
  open RuntimeExpr
  infixl 5 _,_
  data Context : Set where
    ∅ : Context
    _,_ : Context → Type → Context

  data TContext : Set where
    ∅ : TContext
    _,_ : TContext → Kind → TContext

  push : Kind → TContext → TContext
  push k Δ = Δ , k
  EContext = Σ[ n ∈ ℕ ] Data.Vec.Vec (Type × Effects) n
  pattern ∅' = 0 ,, Data.Vec.[]
```

Most of typing judgement are working as before. We omit ones that show up in surface language, as they just pass extra context throughout.

```
private
  variable
    Γ : Context
    Δ : TContext
    Θ : EContext
    e e1 e2 : RExpr
    A B : Type
    E E' F : Effects
    k : Kind
infix 4 _;_;_⊢_∶_/_
data _;_;_⊢_∶_/_ : TContext → EContext → Context → RExpr → Type → Effects → Set where
```

Since labels are represented by natural numbers, ensuring type of label is just a lookup in apropriate context.

```
⊢label : ∀ {n }
  → Θ ∋l n ∶ A / E
  ------------------------------------------
  → Δ ; Θ ; Γ ⊢ label n ∶ (L (Effect n) at A / E) / F
```

3.2 Runtime embedding

We defined embedding of original language in current one, and proof that such embedding preserves typing judgements. Part of proof is ommited as it goes through almost all judgement types.

```
module Transform where
        open Types.Typing
        open Types.Expr
        open RuntimeExpr
        open Typing
        rt-t : Types.Types.Type → Type --runtime-type
        rt-t' : Types.Types.Effects → Effects
        rt-t (Types.Types.ttv x) = ttv x
        rt-t (x Types.Types.- x₁ > x₂) = rt-t x - rt-t' x₁ > rt-t x₂
        rt-t (Types.Types.forallt k x ef) = forallt k (rt-t x) (rt-t' ef)
        rt-t (Types.Types.L l at x / ef) = L rt-t l at rt-t x / rt-t' ef
        rt-t' nil = nil
        rt-t' (x :: xs) = rt-t x :: rt-t' xs
        runtime : Types.Expr.Expr → RExpr
        runtime (var x) = var x
        runtime (lam x) = lam (runtime x)
        runtime (app x x₁) = app (runtime x) (runtime x₁)
        runtime (tlam x x₁) = tlam x (runtime x₁)
        runtime (tapp x x₁) = tapp (runtime x) (rt-t x₁)
        runtime (new x) = new (runtime x)
        runtime (shift₀ x x₁) = shift₀ (runtime x) (runtime x₁)
        runtime (reset₀ x x₁ x₂) = reset₀ (runtime x) (runtime x₁) (runtime x₂)

        runtimeΔ : Types.Typing.TContext → Typing.TContext
        runtimeΔ ∅ = ∅
        runtimeΔ (Δ , k) = runtimeΔ Δ , k
        runtimeΓ : Types.Typing.Context → Typing.Context
        runtimeΓ ∅ = ∅
        runtimeΓ (Γ , T) = runtimeΓ Γ , rt-t T
```

```
runtime-types : ∀ {Δ Γ e T E}
  → Δ Types.Typing.⊢ Γ ⊢ e ꞉ T / E → (runtimeΔ Δ ; ∅' ; runtimeΓ Γ ⊢ (runtime e) ꞉ rt-t T / rt-t' E)

runtime-types (⊢var x) = ⊢var ( runtime∋ x )
runtime-types (⊢lam t) = ⊢lam (runtime-types t)
runtime-types (⊢weak x x₁ t) = ⊢weak (runtime<t꞉ x) (runtime<꞉ x₁) (runtime-types t)
runtime-types (⊢app t t₁) = ⊢app (runtime-types t) (runtime-types t₁)
runtime-types (⊢forall t) = ⊢forall (rt-bump (runtime-types t))
runtime-types (⊢tapp {A = A} x t) = rt-tapp {A = A} (⊢tapp (runtime⊢t x) (runtime-types t))
runtime-types (⊢new t) = ⊢new ( rt-bump( runtime-types t))
runtime-types (⊢shift₀ x t t₁) = ⊢shift₀ ((runtime⊢e x)) (runtime-types t) (runtime-types t₁)
runtime-types (⊢reset₀ x t t₁ t₂) = ⊢reset₀ (runtime⊢e x) (runtime-types t₂) (runtime-types t) (runtime-types t₁)
```

Small lemma about lifting context and that it keeps type safety.

```
open Typing
_⧺_ : Typing.Context → Typing.Context → Typing.Context
y ⧺ ∅ = y
y ⧺ (xs , x) = (y ⧺ xs) , x
∋↑ : ∀ {Γ x A Γ'}
     → Γ ∋ x ꞉ A
     → Γ' ⧺ Γ ∋ x ꞉ A
e↑ : ∀ {Δ Θ Γ e T E Γ'}
     → Δ ; Θ ; Γ ⊢ e ꞉ T / E
     → Δ ; Θ ; (Γ' ⧺ Γ) ⊢ e ꞉ T / E
```

Substitution and proof of type preservation in substitution are defined at the same time.

```
module RExprSubstTyped where
  open RuntimeExpr
  ext : ∀ {Γ Γ' }
      → (∀ {A n } → Γ ∋ n ꞉ A → Σ[ m ∈ ℕ ] Γ' ∋ m ꞉ A)
      → (∀ {A B n} → (Γ , B) ∋ n ꞉ A → Σ[ m ∈ ℕ ] (Γ' , B) ∋ m ꞉ A)
  rename : ∀ {Γ Γ'}
      → (∀ {A n } → Γ ∋ n ꞉ A → Σ[ m ∈ ℕ ] Γ' ∋ m ꞉ A)
      → (∀ {Δ Θ A e E} → Δ ; Θ ; Γ ⊢ e ꞉ A / E → Σ[ e' ∈ RExpr ] Δ ; Θ ; Γ' ⊢ e' ꞉ A / E)
  exts : ∀ {Γ Γ' Δ Θ}
      → (∀ {n A E} → Γ ∋ n ꞉ A → Σ[ e ∈ RExpr ] Δ ; Θ ; Γ' ⊢ e ꞉ A / E)
      → (∀ {n A B E} → Γ , B ∋ n ꞉ A → Σ[ e ∈ RExpr ] Δ ; Θ ; Γ' , B ⊢ e ꞉ A / E)
```

```
subst : ∀ {Δ Γ Γ' Θ}
    → (∀ {n A E} → Γ ∋ n ⦂ A → Σ[ e ∈ RExpr ] Δ ; Θ ; Γ' ⊢ e ⦂ A / E)
    → (∀ {e A E} → Δ ; Θ ; Γ ⊢ e ⦂ A / E → Σ[ e ∈ RExpr ] Δ ; Θ ; Γ' ⊢ e ⦂ A / E)


_[_] : ∀ {Δ Θ Γ A B E1}
    → (e e1 : RExpr)
    → {te : Δ ; Θ ; Γ , A ⊢ e ⦂ B / E1}
    → {te1 : ∀ {E } → Δ ; Θ ; Γ ⊢ e1 ⦂ A / E}
    → Σ[ e' ∈ RExpr ] Δ ; Θ ; Γ ⊢ e' ⦂ B / E1
_[_] {Δ}{Θ}{Γ}{A}{B}{E1}e e1 {te}{te1} = subst {Δ}{Γ , A}{Γ} σ te
    where
    σ : (∀ {B n E} → Γ , A ∋ n ⦂ B → Σ[ e ∈ RExpr ] Δ ; Θ ; Γ ⊢ e ⦂ B / E)
    σ {E = E'} Z = e1 ,, (te1 {E'})
    σ {n = suc n} (S x) = var n ,, ⊢var x
```

Chapter 4

Progress

4.1 Values

Here we define values. Only abstractions, type abstractions and labels are considered values. Since values themself don't perform any effects, they have `nil` effect. But rules for all of them have built-in weakinging. We can use that to generalize their type and perform substiution where any effect is expected.

```
data Value : RExpr → Set where
  vlam : ∀ { e } → Value (lam e)
  vLam : ∀ { k e } → Value (tlam k e)
  vlab : ∀ { n } → Value (label n)
gvalue : ∀ {Δ Θ Γ T E e} → (Value e) → (Δ ; Θ ; Γ ⊢ e ꞉ T / E) → ∀ {F} → (Δ ; Θ ; Γ ⊢ e ꞉ T / F)
gvalue vlam (⊢lam t) = ⊢lam t
gvalue vLam (⊢forall t) = ⊢forall t
gvalue vlab (⊢label x) = ⊢label x
gvalue {Δ} v (⊢weak x x₁ x₂) {F} = (⊢weak x ( <꞉e-refl {Δ} {F}) (gvalue v x₂))
```

Frames would usally be named evaluation context, but here it's taken by typing context. Here frame represents parts between reset s, or reset and shift . Frame type is parametrized by Θ Γ - typing context outside of frame, T type of the hole. It's also indexed by Effects and Type of whole frame, Effects of the hole, typing context of the hole and amount of new' constructors - which says how many type binders are in the frame. Frames here are intrinsically typed, thus they also store type judgements of subexpressions.

```
data Frame (Θ : EContext) (Γ : Context) (T : Type) : Effects → Type → Effects → Set where
  fempty : ∀ {Eff}
  ----------------------------
    → Frame Θ Γ T Eff T Eff
  fapp₁ : ∀ {A B Eff E} → Frame Θ Γ T Eff (A - Eff > B) E
    → (e : RExpr) → { ∅ ; Θ ; Γ ⊢ e ꞉ A / Eff }
    -------------------------
    → Frame Θ Γ T Eff B E
  fapp₂ : ∀ {A B Eff E} → (e : RExpr) → { v : Value e}
    → { ∅ ; Θ ; Γ ⊢ e ꞉ ( A - Eff > B) / Eff }
    ------------------------------------------------------------
    → Frame Θ Γ T Eff A E → Frame Θ Γ T Eff B E
  freset-label : ∀ {A E A' Eff C}
    → (e en : RExpr)
    → ∅ ; Θ ⊢ C ꞉e
    → ∅ ; Θ ; Γ  ⊢ e ꞉ A' / (C :: Eff)
    → ∅ ; Θ ; (Γ , A')  ⊢ en ꞉ A / Eff
    → Frame Θ Γ T nil (L C at A / Eff) E
```

```
    -------------------------
    → Frame Θ Γ T Eff A E
  fshift-label : ∀ {A E A' E' C}
    → (e : RExpr)
    → ∅ ; Θ ⊢ C ∋e
    → ∅ ; Θ ; (Γ , A - E' > A' ) ⊢ e ∋ A' / E'
    → Frame Θ Γ T nil (L C at A' / E') E
    -------------------------------------
    → Frame Θ Γ T (C :: nil) A E
```

Frame plugging and composition:

```
plug : ∀ {Θ Γ T Eff A E}
  → Frame Θ Γ T Eff A E
  → (e : RExpr) → ∅ ; Θ ; Γ ⊢ e ∋ T / E
  → Σ[ res ∈ RExpr ] (∅ ; Θ ; Γ ⊢ res ∋ A / Eff)
_∘f_ : ∀ {Γ Θ Eff Eff' Eff'' A B C }
  → Frame Θ Γ B Eff A Eff'
  → Frame Θ Γ C Eff' B Eff''
  → Frame Θ Γ C Eff A Eff''
```

We can prove how plugging and composition relate. Plugging expression into one frame and another results in same value and type as plugging expression into composition of two frames.

```
∘f-lemma : ∀ {Γ Θ Eff Eff' Eff'' A B C }
  → (f1 : Frame Θ Γ B Eff A Eff' )
  → (f2 : Frame Θ Γ C Eff' B Eff'' )
  → (e : RExpr) → (t : ∅ ; Θ ; Γ ⊢ e ∋ C / Eff'')
  → plug ( f1 ∘f f2) e t
  ≡ ((λ x → plug f1 (Data.Product.proj₁ x) (Data.Product.proj₂ x))(plug f2 e t))

∘f-lemma fempty f2 e t = refl
∘f-lemma (fapp₁ f1 e₁) f2 e t rewrite ∘f-lemma f1 f2 e t = refl
∘f-lemma (fapp₂ e₁ f1) f2 e t rewrite ∘f-lemma f1 f2 e t = refl
∘f-lemma (freset-label ee en x x₁ x₂ f) f2 e t rewrite ∘f-lemma f f2 e t = refl
∘f-lemma (fshift-label e₁ x x₁ f) f2 e t rewrite ∘f-lemma f f2 e t = refl
```

Lifting frames into arbitrary context preserves types, same as with expressions.

```
↑f : forall { Θ A B Eff Eff' Γ' Γ}
  → Frame Θ Γ A Eff B Eff'
  → Frame Θ (Γ' ++ Γ) A Eff B Eff'
```

Metaframe stores whole evaluation context, it's split into frames separated by resets. Type parameters and indices work the same as in frame. Unlike in frame, metaframe now stores resets, so lists of effects inside and outside of frame may differ. That means their difference represents list of effects handled by the frame. This observation can be used to prove that for well typed expression (in empty typing context, and with condition that same labels have the same type) that decomposes into metaframe and `shift₀` expression, and metaframe should handle effect of the `shift`. Also this metaframe decomposes into two metaframes separated by `reset₀` which has same label as mentioned shift.

```
data Metaframe (Θ : EContext) (Γ : Context) (T : Type) (Eff : Effects)
  : Type → Effects → Set where
  mfempty : Metaframe Θ Γ T Eff T Eff
  mfreset : ∀ { A B C Eff' }
    → (l : Label)
    → ∅ ; Θ ⊢ C ∋e
    → ∅ ; Θ ; Γ ⊢ label l ∋ (L C at B / Eff) / nil
    → (e : RExpr) → (∅ ; Θ ; Γ , A ⊢ e ∋ B / Eff)
    → Metaframe Θ Γ T (C :: Eff) A Eff'
```

```
    ---------------------------------
    → Metaframe Θ Γ T Eff B Eff'
  mframe : ∀ {A Eff' B Eff'' }
    → Frame Θ     Γ A Eff B Eff'
    → Metaframe Θ Γ T Eff' A Eff''
    ----------------------------------------
    → Metaframe Θ Γ T Eff B Eff''
```

Metaframes, same as frames, can be lifted into arbitrary contexts, plugged and composed. They can also be composed with simple frames.

```
↑m : forall {Θ A B Eff Eff' Γ' Γ}
  → Metaframe Θ Γ       A Eff B Eff'
  → Metaframe Θ (Γ' ⧺ Γ) A Eff B Eff'
mplug : ∀ {Θ Γ T Eff A E}
  → Metaframe Θ Γ T Eff A E
  → (e : RExpr) → ∅ ; Θ ; Γ ⊢ e ꞉ T / E
  → Σ[ res ∈ RExpr ] (∅ ; Θ ; Γ ⊢ res ꞉ A / Eff)
_∘m_ : ∀ {Γ Θ Eff Eff' Eff'' A B C }
  → Metaframe Θ Γ B Eff A Eff'
  → Metaframe Θ Γ C Eff' B Eff''
  → Metaframe Θ Γ C Eff A Eff''
_f∘m_ : ∀ {Γ Θ Eff Eff' Eff'' A B C }
  → Frame Θ Γ B Eff A Eff'
  → Metaframe Θ Γ C Eff' B Eff''
  → Metaframe Θ Γ C Eff A Eff''
```

4.2 Reduction

Since labels need to be allocated, reduction relation is defined in terms of expression and state. State itself is just next label to be allocated. As frames are intrinsically typed, we need to provide judgment representing well-typedness of expressions.

```
infix 2 _↦_
State = EContext
data _↦_ : RExpr × State → RExpr × State → Set where
  {-
 ↦new : ∀ {e Δ Θ E T A1 E1}
  → {te : (Δ , Kind.E) ; Θ ;(∅ , L ttv zero at A1 / E1)  ⊢ e ꞉ E / T}
  → new e ,′ Θ  ↦  RExprSubstTyped._[_] e (label (Θ .proj₁)) {te = te} {te1 = {!!}} .proj₁  ,′ Θ --(Data.Vec._++_ Θ (  Data.Vec
 -}
  β-lam-app : ∀ {e V Θ Γ A B E }
    → {te : ∅ ; Θ ; (Γ , A) ⊢ e ꞉ B / E}
    → {tv : ∅ ; Θ ; Γ ⊢ V ꞉ A / E}
    → Value (lam e)
    → (v : Value V)
    → app (lam e) V ,′ Θ ↦ (proj₁ (RExprSubstTyped._[_] e V {te = te} {te1 = (gvalue v tv)})) ,′ Θ

  β-tlam-tapp : ∀ {k e T Θ}
    → Value (tlam k e)
    → tapp (tlam k e) T ,′ Θ ↦ e RExprSubst.e[t T ] ,′ Θ

  reset₀-vl : ∀ {V e' en Θ Γ A B E }
    → ( v : Value V)
    → {tv : ∅ ; Θ ; Γ ⊢ V ꞉ A / E}
    → {ten : ∅ ; Θ ; (Γ , A) ⊢ en ꞉ B / E}
    → reset₀ V en e' ,′ Θ ↦ (RExprSubstTyped._[_] en V {te = ten} {te1 = (gvalue v tv)} .proj₁) ,′ Θ

{-
 reset₀-k : ∀ {es en e' e s n Θ A T Eff Eff' B A' E' ls lr}
   → { f : Metaframe Θ ∅ T Eff A Eff'  }
   → ∀ {cont-type}
   → Value (e')
   --shift
   → {ts : ∅ ; Θ ; ∅ ⊢ (shift₀ e' es) ꞉ T / Eff'}
   → {tes : ∅ ; Θ ; (∅ , T - Eff' > B) ⊢ es ꞉ B / Eff' }
   → {tls : ∅ ; Θ ; ∅ ⊢ e' ꞉ (L ttv ls at B / Eff') /  nil }
   → {tlvs : ∅ ; Θ  ⊢  ttv lr ꞉e }
   --reset
   → {te : Δ ; ∅ ⊢ e ꞉ A' / (ttv lr :: E') }
```

```
  → {ten : Δ ; ∅ , A' ⊢ en ꞉ T / Eff }
  → {tlr : Δ ; ∅ ⊢ e' ꞉ (L ttv lr at T / Eff) /  nil }
  → {tlvr : Δ  ⊢  ttv lr ꞉e }
  → (proj₁ (mplug  f (shift₀ e' es) ts)) ≡ e
  → reset₀ e en e' ,′ s
    ↦ RExprSubstTyped._[_] es
      (lam (reset₀ (proj₁ (mplug (↑m {Γ' = ∅ , T} f) (var 0) (⊢var Z))) en e'))
      {te = tes} {te1 = gvalue {E = Eff} vlam cont-type}
      .proj₁  ,′ s
-}
```

Since simple reduction above is defined directly on redexes, we introduce -→ that represents reduction within metaframe. As we are only considering whole typed expressions, in place of Γ we use empty context.

```
{-
infix 2 _-→_
data _-→_ : RExpr × State → RExpr × State → Set where
  -→frame : ∀ {e1 e1' e2 e2' s s' n Δ Δ' A T Eff Eff' t1 t2} → (f : Metaframe Δ ∅ T Eff A Eff' Δ' n)
    → e1' ,′ s ↦ e2' ,′ s'
    → Data.Product.proj₁ (mplug f e1' t1) ≡ e1
    → Data.Product.proj₁ (mplug f e2' t2) ≡ e2
    →  (e1 ,′ s) -→ (e2 ,′ s')
-}
```

4.3 Progress

```
{-
data Decompose : State → (e : RExpr)  → Set where
  de-simpl-redex : ∀ {e e2 s s' n Δ Δ' A T Eff Eff'}
    → (f : Metaframe Δ ∅ T Eff A Eff' Δ' n)
    → (e ,′ s) -→ (e2 ,′ s')
    → (t : Δ ; ∅ ⊢ e ꞉ A / Eff)
    → Decompose s e
  de-shift : ∀ {s Δ Δ' T Eff A n Eff' es es' e l t}
    → (f : Metaframe Δ ∅ T Eff A Eff' Δ' n)
    →  shift₀ (label l) es' ≡ es
    → Data.Product.proj₁ (mplug f es t) ≡ e
    → (t : Δ ; ∅ ⊢ e ꞉ A / Eff)
    → (ts : Δ ; ∅ ⊢ es ꞉ T / Eff')
    → Decompose s e
  de-val : ∀ {Δ Eff A s e} → { t : Δ ; ∅ ⊢ e ꞉ A / Eff}
    → Value e
    → Decompose s e
data Progress : State → RExpr → Set where
  done : ∀ {e s} →  Value e → Progress s e
  step : ∀ {e1 s1 e2 s2}
    → ( e1 ,′ s1 ) -→ ( e2 ,′ s2)
    → Progress s1 e1
-}
```

Proof of progress would have a type of `progress : ∀ {A Δ Effs} → (s : State) → (e : RExpr) → (t : Δ ; ∅ ⊢ e ꞉ A / Effs) → Progress s e`. In such proof We would use auxiliary struct `Decompose` which builder would walk down well typed expression recursively until it has reached either value, simple reduction (app, tapp, new), or shift, and return it with surrounding metaframe.

In case of shift, such metaframe by construction should have effect handler that has same effect as shift. So we can construct `rest₀-k` and surrounding metaframe. Other cases would either be immediate value, or simple reduction in context.

4.3.1 Preservation

Current definition of reduction relation would yield proof of preservation immediately if progress was given since, frames and expressions are well typed, and reduction relation requires proofs to plug into metaframes or subsitution.

Chapter 5

Discussion/Closing thoughts

In this paper we presented language with labeled delimited effect handlers together with partial formalisation of such language. We defined 3 term languages, typing judgements for 2 of them, translations between those languages and that typing is preserved. We defined intrinsically typed frames and metaframes that allow us to define reduction relation, how it preserves types. Since they are typed we statically know what effects are handled by frame. We also defined draft of abstract machine and how it could evaluate this language. This formalisation could be expanded and finished to fully prove correctness of the language and translations.

Bibliography

[1] Guy Lewis Steele, Gerald Jay Sussman. The Revised Report on SCHEME: A Dialect of LISP. In MIT Artificial Intelligence Memo 452, 1978. URL: https://dspace.mit.edu/handle/1721.1/6283.

[2] Olivier Danvy, Andrzej Filinski. Abstracting control. In LISP and Functional Programming, pages 151-160, 1990. URL: https://dl.acm.org/doi/10.1145/91556.91622

[3] Matthias Felleisen. The theory and practice of first-class prompts. In POPL '88: Proceedings of the 15th ACM SIGPLAN-SIGACT symposium on Principles of programming languages, pages 180-190, 1988. URL: https://dl.acm.org/doi/10.1145/73560.73576

[4] Maciej Piróg, Piotr Polesiuk, Filip Sieczkowski. Typed Equivalence of Effect Handlers and Delimited Control. In 4th International Conference on Formal Structures for Computation and Deduction (FSCD 2019), pages 30:1-30:16, 2019. URL: https://doi.org/10.4230/LIPIcs.FSCD.2019.30

[5] Marek Materzok, Dariusz Biernacki. Subtyping Delimited Continuations. In ACM SIGPLAN Notices, Volume 46, Issue 9, pages 81-93, 2011. URL: https://dl.acm.org/doi/abs/10.1145/2034574.2034786